

Part 1: AI Engineer Assessment Answers

Q1. Design the architecture for this system.

Answer:

My architecture is an event-driven "Agentic Reflexion" Microservice, designed to decouple heavy AI workloads from the main web server to prevent event-loop blocking.

- **Input Layer:** A FastAPI gateway accepting RESTful payloads (lead forms/chat).
- **LLM Usage:** Local inference using Ollama (Llama 3) for the MVP to guarantee 100% data privacy and zero API costs.
- **Classification & Response (The Logic):** Orchestrated via a **LangGraph State Machine**. It uses a multi-agent loop:
 1. *Semantic Evaluator:* A gatekeeper that filters out spam/low-density input.
 2. *Memory Retriever:* Pulls RAG context from the vector database.
 3. *Worker Agent:* Classifies the lead (Hot/Warm/Cold) and drafts the response.
- **Fallbacks:** A **Validator Agent** reviews the Worker's draft. If it detects hallucinations, it loops the state back for a rewrite (capped at 2 retries).
- **Storage:** ChromaDB for vector memory, PostgreSQL for transactional logging.
- **Queue:** Currently managed via Python `asyncio`.
- **Scaling Roadmap:** This architecture is governed by VARYNT's budget and infra constraints. As active user usage and read/write IOPS increase, we will transition the `asyncio` queue to a

dedicated Redis/RabbitMQ broker, and swap the local Llama 3 model for a managed cloud cluster to handle high concurrency.

Q2. Implement ONE core part (your choice).

Answer:

Selected: AI pipeline (input → output).

Implementation Details: I implemented the complete Multi-Agent LangGraph pipeline ([lead_pipeline.py](#)). Instead of a brittle, single-shot LLM prompt, I built a [StateGraph](#) that manages a "Reflexion Loop" (Actor-Critic pattern). It proves the system can evaluate intent, retrieve semantic memory, draft a personalized email, and self-correct its own hallucinations before returning the final JSON payload to the FastAPI router.

Q3. Provide Prompts & Quality Control

Answer:

- **Classification Prompt:** *"You are a KeaBuilder CRM routing agent. Classify the lead intent (HOT/WARM/COLD) and draft a brief, personalized email. Historical Context: {context}. Feedback: {feedback}. Output EXACTLY this JSON format..."*
- **Response Prompt (Validator):** *"Review this drafted email against the original user lead. Rules: 1. Did it directly address the user's specific goal? 2. Is the tone professional? 3. Did it hallucinate pricing? If it passes, approve it. If it fails, provide explicit rewrite instructions."*
- **Quality Control (Generic/Hallucinated/Context-Aware): ***
Context-Awareness: Achieved via the ChromaDB Retriever agent injecting historical twins into the prompt.
 - *Anti-Hallucination & Specificity:* The Validator Agent acts as a supervisor. By explicitly instructing the Worker to extract

the user's specific "entity" (e.g., 'gym funnel'), we prevent generic outputs.

- **Scaling Roadmap:** As the budget permits, we will upgrade these JSON-enforced prompts to utilize native OpenAI Structured Outputs (Function Calling) for near-100% deterministic schema adherence.

Q4. Explain how your system handles edge cases.

Answer:

- **Low/Garbage input:** Handled by the *Evaluator Agent*. If semantic density is too low, the graph terminates immediately, saving compute costs and returning a standard clarification prompt.
- **Ambiguous leads:** Classified as WARM by the Worker, which is prompted to generate a clarifying, question-based email draft.
- **Model/API failure:** Handled via standard FastAPI `try/except` blocks returning a 502/202 status.
- **Incorrect classification:** Caught by the *Validator Agent*, which triggers a graph cycle back to the Worker to fix the classification before the user sees it.
- **Scaling Roadmap:** For high-volume production, I will implement Circuit Breaker patterns. If the primary LLM times out due to infrastructure load, the system will seamlessly failover to a highly quantized, local 3B parameter model to guarantee an unbroken user experience.

Q5. How will you monitor this system?

Answer:

- **Current MVP:** I built a custom Python `logging` tracing layer (`varynt_agent_trace.log`) that records every graph node transition, agent thought, and validation critique for local auditing.

- **Scaling Roadmap:** Monitoring is strictly tied to operational budget. As we move to production, I will integrate **LangSmith** for granular LLM observability (latency, token costs, hallucination rates) and **Datadog/Prometheus** to monitor the FastAPI container health and memory usage.

Q6. What trade-offs did you make while designing this system?

Answer:

- **Accuracy vs. Latency:** I implemented a LangGraph Reflexion Loop, which trades a few seconds of execution time for a massive increase in response accuracy and safety. A delayed accurate response is vastly superior to an instant hallucination.
 - **Cost vs. Capability:** I chose local **all-MiniLM** embeddings and local **Llama 3** over paid APIs. This requires more local compute architecture but keeps the operational budget at zero during the MVP phase.
 - **Scaling Roadmap:** This is a "Best-in-Class" MVP. We have the freedom to expose alternate strategies based on VARYNT's real-world metrics. If our infra constraints dictate a need for massive horizontal scaling, we will decouple the local models and route to managed cloud endpoints, spending budget only when the read/write volume necessitates it.
-

Part 2: ML Engineer Assessment Answers

Q1. Build a Mini ML system.

Answer:

I built a localized semantic retrieval engine inside the FastAPI microservice. It utilizes **sentence-transformers** to generate dense vector embeddings from raw text, stores them persistently in a local **ChromaDB** instance, and exposes a **/search** endpoint to retrieve the most similar historical leads using Cosine Similarity.

Q2. Explain your architecture.

Answer:

- **Embedding Model:** **all-MiniLM-L6-v2**. Chosen for its exceptionally low latency and small memory footprint on CPU-only infrastructure.
- **Database:** **ChromaDB** (Local SQLite).
- **API Flow:** FastAPI receives the POST request -> Embeds the text -> Queries ChromaDB -> Returns a JSON array of top matches.
- **Data Pipeline:** "Embed-on-Ingest" pattern. New templates/leads are vectorized immediately upon creation.
- **Scaling Roadmap:** As the system approaches 100k+ records, local SQLite becomes an I/O bottleneck. Based on our infrastructure budget, I will migrate this to a managed **Pinecone** or **Milvus** cluster to distribute the vector load.

Q3. How will you optimise retrieval speed? Handling large-scale data?

Answer:

- **Optimization & Indexing:** The system utilizes **HNSW (Hierarchical Navigable Small World)** indexing natively through ChromaDB. This graph-based index allows for approximate nearest neighbor (ANN) search with sub-millisecond latency, offering an $O(\log N)$ search complexity.
- **Scaling Roadmap:** For 100k+ records, maintaining HNSW entirely in RAM becomes expensive. I would implement metadata pre-filtering (e.g., filtering by **industry** or **date** before vector search) to reduce the search space, and shard the database across multiple nodes.

Q4. How will your system handle failures?

Answer:

- **Invalid input:** Blocked at the API layer using strict **Pydantic** validation schemas.
- **Missing embeddings:** A background cron job / async worker will regularly scan the PostgreSQL relational DB for missing vector IDs and re-embed them.
- **Slow model responses:** Implementing an Async Webhook pattern. The UI receives an HTTP 202 immediately, and the final search results are pushed via WebSockets once the embedding is generated.
- **Database failures:** Local ChromaDB is backed by SQLite file persistence. In production, we will use automated volume backups and a multi-AZ deployment strategy.

Q5. What trade-offs did you make?

Answer:

- **Dimensionality vs. Granularity:** I chose a 384-dimensional model (MiniLM) rather than a 1536-dimensional model (like OpenAI's `text-embedding-3`). This trades a fractional amount of semantic nuance for massive savings in storage space and compute overhead, aligning with our current local-first infrastructure constraints.
- **Scaling Roadmap:** I avoided over-engineering a distributed cloud vector DB on day one. We will implement heavy-lifting databases only when the active usage and read/write metrics dictate the necessity, ensuring we don't rush to buy infrastructure we don't yet need.

Q6. How will you evaluate if your similarity system is working correctly?

Answer:

- **Current MVP:** I use a threshold-based Cosine Similarity metric. If top results consistently fall below a specific confidence distance, the model is flagged for review. I also utilize a manual "golden dataset" of known lead-pairs to test retrieval accuracy.
- **Scaling Roadmap:** In production, I will implement a user-feedback loop in the KeaBuilder UI (e.g., a simple "thumbs up/down" on suggested templates). This telemetry will allow us to continuously fine-tune the embedding model weights using a triplet-loss function based on actual user behavior.

Conclusion & Scaling Roadmap: The "Modular Realism" Approach

This MVP demonstrates a production-aware AI Microservice that goes beyond simple LLM wrappers. By implementing a **LangGraph Reflexion Loop** and **ChromaDB Vector Memory**, the system achieves self-correction, high context-awareness, and strict hallucination control.

However, a truly senior architecture is defined not just by what it can do today, but by how it adapts tomorrow. This system was built on the philosophy of **Modular Realism**—prioritizing 100% data privacy and zero API costs for the MVP via local models (**Llama 3** and **all-MiniLM-L6-v2**), while keeping the business logic completely decoupled from the infrastructure layer.

As VARYNT transitions from MVP to enterprise-scale, we will monitor specific constraints and trigger the following scaling strategies:

1. Infrastructure Scaling (Based on Active Usage & Concurrency)

- **Current State:** Handled via FastAPI **asyncio** background tasks.
- **The Pivot:** When active user concurrency spikes and threatens API response times, we will inject a dedicated message broker (**RabbitMQ** or **Redis**). This will decouple the ingestion layer from the heavy AI processing layer, allowing us to spin up multiple Dockerized worker nodes behind a load balancer to handle the queue.

2. Database Scaling (Based on Read/Write Volume)

- **Current State:** Local SQLite-backed ChromaDB. Highly efficient for sub-10k records with zero hosting costs.

- **The Pivot:** As lead intelligence data and funnel templates surpass 100k+ records, the local I/O will bottleneck. Based on the operational budget, we will migrate the vector store to a distributed, managed cluster like **Pinecone** or **Milvus**, enabling metadata sharding and high-throughput vector retrieval.

3. Compute & Budget Scaling (Based on Model Performance)

- **Current State:** Local inference via Ollama to maximize runway and ensure strict data governance.
- **The Pivot:** Once VARYNT has a defined API budget and requires highly complex reasoning at high speeds, we will seamlessly swap the local LLM endpoint for managed cloud providers (e.g., OpenAI, Anthropic) or deploy our own quantized models on dedicated GPU clusters via **vLLM**. The LangGraph architecture remains identical; only the engine changes.

Final Note: Building for production means knowing when to be scrappy and when to scale. This architecture gives **VARYNT** the absolute best of both worlds. I am incredibly excited about the opportunity to bring this vision to life at **Dream Reflection Media** and develop this product end-to-end. Thank you for your time and consideration!

- By Sri Bharath Pentela
AI Engineer | Cognizant
Technology Solutions

sribharathpentelasvm@gmail.com

+91 8125639020.

